Pasos para Modelos de Regresión

Desde la preparación hasta la implementación

9 de marzo de 2026

Resumen

Este documento presenta un temario exhaustivo para el desarrollo de modelos de regresión, cubriendo desde la preparación específica post-EDA hasta la implementación en producción. Incluye técnicas avanzadas, validación robusta, interpretabilidad y consideraciones prácticas para casos reales.

Índice

1. Preparación Específica para Regresión **3**
1.1. Verificaciones Post-EDA . . . 3
1.2. Transformaciones Específicas para Regresión . . . 3

2. Tipos de Modelos de Regresión **3**
2.1. Modelos Paramétricos . . . 3
2.2. Modelos No Paramétricos y Basados en Árboles . . . 4
2.3. Modelos Avanzados . . . 4

3. Preprocesamiento y Pipeline **5**
3.1. Pasos Obligatorios . . . 5
3.2. Pipeline de Modelado . . . 5

4. Validación y Evaluación **5**
4.1. Métricas de Evaluación . . . 5
4.2. Estrategias de Validación . . . 6
4.3. Análisis de Residuos . . . 6

5. Selección de Modelos y Optimización **7**
5.1. Búsqueda de Hiperparámetros . . . 7
5.2. Comparación de Modelos . . . 7

6. Interpretabilidad y Explicabilidad **8**
6.1. Importancia de Variables . . . 8
6.2. Métodos Globales . . . 8
6.3. Métodos Locales . . . 8
6.4. Casos Específicos de Interpretación . . . 9

7. Problemas Comunes y Soluciones **9**
7.1. Overfitting y Underfitting . . . 9
7.2. Heterocedasticidad . . . 9
7.3. Outliers y Puntos Influyentes . . . 10
7.4. Colinealidad Perfecta . . . 10

8. Técnicas Avanzadas **10**
8.1. Regresión Robusta . . . 10
8.2. Regresión con Series Temporales . . . 10
8.3. Ensamblado de Modelos . . . 11
8.4. Regresión con Datos Desbalanceados en el Continuo . . . 11

9. Producción y Mantenimiento **11**
9.1. Serialización y Exportación 11
9.2. API y Despliegue 11
9.3. Monitoreo en Producción 12

10. Checklist de Implementación **12**

11. Conclusiones y Buenas Prácticas **13**

1 Preparación Específica para Regresión

1.1 Verificaciones Post-EDA

- **Distribución de la variable objetivo:**
  - Análisis de asimetría (skewness) en y
  - Transformaciones recomendadas: $\log(y)$, $\sqrt{y}$, Box-Cox, Yeo-Johnson
  - Detección de outliers en variable objetivo
  - Verificar si es realmente continua o es discreta con muchos valores
- **Relaciones lineales vs no lineales:**
  - Matriz de correlación con y
  - Scatter plots parciales (partial dependence plots preliminares)
  - Test de linealidad (Rainbow test, Harvey-Collier)
- **Multicolinealidad:**
  - Calcular VIF (Variance Inflation Factor) para cada predictora
  - Umbral común: VIF ¿5-10 indica multicolinealidad severa
  - Matriz de correlación entre predictoras

1.2 Transformaciones Específicas para Regresión

- **Transformaciones de predictoras:**
  - Polinómicas: x^2, x^3 para capturar curvatura
  - Splines y bases polinómicas (regresión spline)
  - Términos de interacción: $x_1 \cdot x_2$, x_1/x_2
  - Transformaciones no lineales: log, exp, recíproco
- **Creación de features específicas:**
  - Dummies para puntos de quiebre (piecewise regression)
  - Variables de cambio de pendiente
  - Componentes estacionales (si aplica)

2 Tipos de Modelos de Regresión

2.1 Modelos Paramétricos

- **Regresión Lineal Múltiple:**
  - Supuestos: linealidad, independencia, homocedasticidad, normalidad
  - Fórmula: $\hat{y} = \beta_0 + \beta_1 x_1 + ... + \beta_p x_p$
  - Estimación por Mínimos Cuadrados Ordinarios (OLS)
- **Regresión Ridge (L2):**
  - Penalización: $\lambda \sum \beta_j^2$
  - Reduce coeficientes pero no los hace cero
  - Útil para multicolinealidad
- **Regresión Lasso (L1):**
  - Penalización: $\lambda \sum |\beta_j|$
  - Realiza selección de variables (coeficientes exactamente cero)

    - Ideal para datasets con muchas variables

- **Elastic Net:**
    - Combina L1 y L2: $\lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$
    - Mejor que Lasso cuando hay grupos de correlación

2.2 Modelos No Paramétricos y Basados en Árboles

- **Árboles de Regresión (CART):**
    - Particionamiento recursivo
    - Criterio: reducción de MSE
    - Propenso a overfitting
- **Random Forest Regressor:**
    - Bagging + aleatoriedad en features
    - Maneja no linealidades automáticamente
    - Importancia de variables incorporada
    - No extrapola bien fuera del rango de entrenamiento
- **Gradient Boosting Machines:**
    - XGBoost, LightGBM, CatBoost
    - Construcción secuencial de árboles
    - Estado del arte en datos tabulares
    - Hiperparámetros críticos: learning rate, n_estimators, max_depth

2.3 Modelos Avanzados

- **SVR (Support Vector Regression):**
    - Kernel trick para relaciones no lineales
    - Sensible a escala de variables
    - Bueno para dimensiones moderadas
- **Redes Neuronales para Regresión:**
    - MLP (Perceptrón Multicapa)
    - Función de activación lineal en capa de salida
    - Loss function: MSE o MAE
    - Requiere muchos datos y escalado
- **Modelos Probabilísticos:**
    - Regresión Bayesiana
    - Gaussian Process Regression
    - Proporcionan incertidumbre en predicciones

3 Preprocesamiento y Pipeline

3.1 Pasos Obligatorios

- **Separación de datos:**
  - Train (60-70 %), Validation (15-20 %), Test (15-20 %)
  - Mantener distribución temporal si es serie de tiempo
  - Estratificación por cuantiles de y si es necesario
- **Escalado de variables:**
  - `StandardScaler` para modelos lineales, SVM, redes
  - `MinMaxScaler` para redes neuronales
  - `RobustScaler` si hay outliers
  - Ajustar SOLO con train, transformar validación y test
- **Manejo de categóricas:**
  - One-Hot Encoding para lineales
  - Label Encoding para árboles (opcional)
  - Target Encoding con cross-validation para evitar leakage

3.2 Pipeline de Modelado

- **Creación de Pipeline en sklearn:**

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(), categorical_features)
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', Ridge(alpha=1.0))
])
```

- **Ventajas:**
  - Evita data leakage
  - Reproducibilidad garantizada
  - Fácil validación cruzada y búsqueda de hiperparámetros

4 Validación y Evaluación

4.1 Métricas de Evaluación

- **Métricas de Error:**
  - **MSE (Mean Squared Error):** $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
  - **RMSE (Root MSE):** $\sqrt{MSE}$ - misma unidad que y
  - **MAE (Mean Absolute Error):** $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
  - **MAPE (Mean Absolute Percentage Error):** $\frac{100\,\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$
  - **sMAPE** (versión simétrica para valores cercanos a cero)

- **Métricas de Bondad de Ajuste:**
  - **R² (Coeficiente de Determinación):** $1 - \frac{SS_{res}}{SS_{tot}}$
  - **R² Ajustado:** $1 - (1 - R)\frac{n-1}{n-p-1}$
  - **Correlación entre y y $\hat{y}$**
- **Métricas Probabilísticas:**
  - Intervalos de predicción
  - Cobertura de intervalos
  - Log-loss para distribuciones

4.2 Estrategias de Validación

- **Validación Cruzada (Cross-Validation):**
  - K-Fold (típicamente k=5 o 10)
  - Stratified K-Fold (para variable objetivo continua estratificada)
  - Time Series Split (para datos temporales)
  - Repeated K-Fold (más estable)
- **Curvas de Validación:**
  - Curvas de aprendizaje (learning curves)
  - Curvas de validación para hiperparámetros
  - Bias-variance tradeoff analysis
- **Bootstrap:**
  - Intervalos de confianza para métricas
  - Estabilidad del modelo

4.3 Análisis de Residuos

- **Gráficos esenciales:**
  - Residuos vs. Predichos (homocedasticidad)
  - Q-Q plot de residuos (normalidad)
  - Residuos vs. Orden de observaciones (independencia)
  - Residuos vs. cada predictora
- **Pruebas estadísticas:**
  - Shapiro-Wilk / Kolmogorov-Smirnov para normalidad
  - Breusch-Pagan / Goldfeld-Quandt para heterocedasticidad
  - Durbin-Watson para autocorrelación
- **Detección de puntos influyentes:**
  - Distancia de Cook
  - Leverage (hat values)
  - DFBETAS, DFFITS

5 Selección de Modelos y Optimización

5.1 Búsqueda de Hiperparámetros

- **Métodos de búsqueda:**
  - Grid Search (exhaustivo pero costoso)
  - Random Search (más eficiente)
  - Bayesian Optimization (Optuna, Hyperopt)
  - Halving Grid Search (sucesivo)
- **Hiperparámetros clave por modelo:**
  - Ridge/Lasso/ElasticNet: α (lambda)
  - Random Forest: `n_estimators, max_depth, min_samples_split`
  - XGBoost: `learning_rate, max_depth, subsample, colsample_bytree`
  - SVR: `C, epsilon, kernel, gamma`
- **Estrategia de búsqueda:**

```python
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'regressor__n_estimators': [100, 200, 300],
    'regressor__max_depth': [3, 5, 7, None],
    'regressor__learning_rate': [0.01, 0.05, 0.1, 0.2]
}

random_search = RandomizedSearchCV(
    pipeline,
    param_distributions=param_dist,
    n_iter=50,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)
```

5.2 Comparación de Modelos

- **Tabla comparativa:**

Modelo	RMSE (Train)	RMSE (Test)	R^2	Tiempo (s)
Regresión Lineal	12.5	13.2	0.82	0.1
Ridge	12.6	12.9	0.83	0.1
Random Forest	6.8	9.5	0.91	5.2
XGBoost	5.2	8.7	0.93	3.8

Cuadro 1: Comparación de modelos de regresión

- **Pruebas estadísticas de comparación:**
  - Test de Diebold-Mariano para predicciones
  - ANOVA de modelos
  - Cross-Validation paired t-test

6 Interpretabilidad y Explicabilidad

6.1 Importancia de Variables

- **Métodos específicos por modelo:**
  - Coeficientes estandarizados (para modelos lineales)
  - `feature_importances_` (para árboles)
  - Permutation importance (model-agnostic)
- **Visualizaciones:**
  - Gráfico de barras de importancia
  - Coeficientes con intervalos de confianza
  - Word clouds de features importantes

6.2 Métodos Globales

- **Partial Dependence Plots (PDP):**
  - Muestran efecto marginal de una variable
  - `sklearn.inspection.plot_partial_dependence`
  - Limitación: asume independencia
- **Accumulated Local Effects (ALE):**
  - Alternativa a PDP sin problema de correlación
  - Más preciso cuando hay correlación
- **SHAP Global:**
  - Valores SHAP promedio por variable
  - Summary plots (bee swarm)
  - Dependence plots

6.3 Métodos Locales

- **LIME (Local Interpretable Model-agnostic Explanations):**
  - Explica predicciones individuales
  - Ajusta modelo simple localmente
- **SHAP (SHapley Additive exPlanations):**
  - Basado en teoría de juegos
  - Consistente y localmente preciso
  - `shap.Explainer`, `shap.TreeExplainer`
- **Contrafactuales:**
  - "¿Qué cambiaría para obtener otra predicción?"
  - Mínimos cambios en features para modificar salida

6.4 Casos Específicos de Interpretación

- **Modelos Lineales:**
  - β_j: cambio en y por unidad de x_j, ceteris paribus
  - Coeficientes estandarizados: $\beta_j \cdot \frac{\sigma_{x_j}}{\sigma_y}$
- **Modelos Log-Log:**
  - $\log(y) = \beta_0 + \beta_1 \log(x_1)$
  - β_1 es elasticidad: 1 % cambio en $x_1 \to \beta_1$ % cambio en y
- **Modelos con interacciones:**
  - Efecto condicional: $\frac{\partial y}{\partial x_1} = \beta_1 + \beta_3 x_2$
  - Visualizar superficie de respuesta

7 Problemas Comunes y Soluciones

7.1 Overfitting y Underfitting

- **Señales de overfitting:**
  - Train error mucho menor que test error
  - Complejidad excesiva del modelo
  - Inestabilidad en pequeñas variaciones de datos
- **Soluciones:**
  - Regularización (L1, L2)
  - Reducir complejidad (menos features, árboles menos profundos)
  - Aumentar datos (data augmentation)
  - Early stopping en boosting
  - Dropout en redes neuronales
- **Señales de underfitting:**
  - Alto error en train y test
  - Modelo demasiado simple
  - No captura patrones evidentes
- **Soluciones:**
  - Aumentar complejidad
  - Crear más features (interacciones, polinomios)
  - Reducir regularización
  - Probar modelos más flexibles

7.2 Heterocedasticidad

- **Detección:** gráfico residuos vs predichos con forma de cono
- **Consecuencias:** errores estándar sesgados, inferencia inválida
- **Soluciones:**
  - Transformar y (log, Box-Cox)
  - Usar mínimos cuadrados ponderados (WLS)
  - Errores estándar robustos (HC3, Huber-White)
  - Modelos que no asumen homocedasticidad (cuantílicos)

7.3 Outliers y Puntos Influyentes

- **Estrategias:**
  - Modelos robustos: HuberRegressor, RANSAC, Theil-Sen
  - Winsorización de la variable objetivo
  - Eliminación solo si hay justificación
  - Modelos basados en árboles (menos sensibles)

7.4 Colinealidad Perfecta

- **Problema:** matrices singulares, coeficientes inestables
- **Soluciones:**
  - Eliminar una variable del par correlacionado
  - Ridge Regression (L2)
  - PCA antes de regresión
  - Interpretar con cautela

8 Técnicas Avanzadas

8.1 Regresión Robusta

- **Huber Regressor:**
  - Combina MSE y MAE
  - Parámetro ϵ controla transición
- **RANSAC (RANdom SAmple Consensus):**
  - Iterativamente selecciona inliers
  - Muy robusto a outliers extremos
- **Quantile Regression:**
  - Predice cuantiles (mediana, percentiles)
  - Útil para intervalos de predicción
  - No asume distribución de errores

8.2 Regresión con Series Temporales

- **Características temporales:**
  - Lags: $y_{t-1}, y_{t-2}, \ldots$
  - Medias móviles
  - Diferencias: $y_t - y_{t-1}$
- **Validación específica:**
  - Time Series Split
  - No usar datos futuros en entrenamiento
- **Modelos especializados:**
  - ARIMA/SARIMA (cuando solo importa y)
  - Prophet (Facebook)
  - LSTM/GRU para secuencias largas

8.3 Ensamblado de Modelos

- **Voting Regressor:**
  - Promedio simple o ponderado de modelos
  - Reduce varianza
- **Stacking:**
  - Meta-modelo aprende de predicciones base
  - Puede mejorar rendimiento significativamente
- **Blending:**
  - Similar a stacking pero con hold-out set
  - Más simple, menos overfitting

8.4 Regresión con Datos Desbalanceados en el Continuo

- **Problema:** regiones del espacio con pocos datos
- **Técnicas:**
  - Muestreo estratificado por cuantiles de y
  - Weighted regression (pesos inversos a densidad)
  - SMOGN (synthetic minority over-sampling technique for regression)
  - Modelos basados en distancias con cuidado

9 Producción y Mantenimiento

9.1 Serialización y Exportación

- **Guardar modelo:**
  - `joblib.dump()` (recomendado para sklearn)
  - `pickle` (pero menos seguro)
  - `ONNX` para interoperabilidad
  - `PMML` para sistemas legacy
- **Registro de versiones:**
  - MLflow
  - DVC (Data Version Control)
  - Comet.ml, Weights & Biases

9.2 API y Despliegue

- **Opciones de despliegue:**
  - REST API con Flask/FastAPI
  - Batch predictions (Airflow, Spark)
  - Streaming (Kafka, AWS Kinesis)
- **Consideraciones:**
  - Tiempo de inferencia
  - Escalabilidad horizontal
  - Monitoreo continuo

9.3 Monitoreo en Producción

- **Métricas de rendimiento:**
  - Seguimiento de RMSE/MAE en tiempo real
  - Alertas cuando empeora
  - Comparación con baseline simple
- **Data drift y concept drift:**
  - Detección de cambios en distribuciones de predictoras
  - Detección de cambios en relación $X \to y$
  - Reentrenamiento programado o por detección
- **Logging y trazabilidad:**
  - Guardar predicciones y features
  - Auditoría de decisiones
  - Explicaciones almacenadas

10 Checklist de Implementación

- **Preparación**
  - ☐ Verificar distribución de y
  - ☐ Calcular VIF para multicolinealidad
  - ☐ Dividir datos (train/validation/test)
- **Modelado Inicial**
  - ☐ Probar modelo baseline (media, mediana)
  - ☐ Probar regresión lineal simple
  - ☐ Probar modelo con regularización
- **Modelos Avanzados**
  - ☐ Random Forest / XGBoost / LightGBM
  - ☐ Optimización de hiperparámetros
  - ☐ Validación cruzada
- **Evaluación**
  - ☐ Calcular múltiples métricas
  - ☐ Analizar residuos
  - ☐ Comparar modelos formalmente
- **Interpretación**
  - ☐ Importancia de variables
  - ☐ SHAP / PDP / ALE
  - ☐ Explicaciones locales para casos test
- **Producción**
  - ☐ Pipeline reproducible
  - ☐ Tests unitarios para transformaciones
  - ☐ API documentation
  - ☐ Sistema de monitoreo

11 Conclusiones y Buenas Prácticas

- **Siempre empezar con modelo simple:** establece baseline
- **Domain knowledge es crucial:** crea features informativas
- **Validación robusta:** no confíes en una sola métrica o split
- **Interpretabilidad vs Performance:** encuentra balance según necesidades
- **Documentación exhaustiva:** cada decisión, cada transformación
- **Pruebas automatizadas:** aseguran calidad continua
- **Feedback loop:** recoge predicciones y resultados reales para mejorar